

数码管扫描显示设计



请帮我根据实验步骤，在vivado中完成实验，写出源代码、仿真激励代码、XDC引脚约束文件，确保我能在板子上跑通。实验名称为Seg7_Scan。



根据您的实验指导书要求，我为您编写了完整的Verilog源代码、仿真测试文件以及适配 HCS-A02 板卡的 XDC 约束文件。

1. 实例先行 (Examples/Code)

请在 Vivado 中创建工程，包含以下三个文件。

(1) 通用数码管显示模块 (Tube.v)

这是底层的驱动模块，负责分时扫描和字形解码。

```
`timescale 1ns / 1ps

module Tube(
    input wire clk,           // 100MHz 系统时钟
    input wire rst,           // 复位信号 (高电平有效)
    input wire [31:0] Data,    // 要显示的32位数据
    output reg [7:0] AN,       // 位选信号 (Active Low)
    output reg [7:0] SEG       // 段选信号 (Active Low)
);

// --- 1. 产生扫描时钟 (约2ms) ---
reg [17:0] count;
wire clk_fresh;

always @(posedge clk or posedge rst) begin
    if(rst)
        count <= 0;
    else
        count <= count + 1'b1;
end

// 100MHz / 2^18 ≈ 381Hz (约2.6ms周期), 符合人眼视觉暂留
assign clk_fresh = count[17];

// --- 2. 位选控制 (轮流点亮) ---
reg [2:0] bit_select;

always @(posedge clk_fresh or posedge rst) begin
    if(rst)
        bit_select <= 0;
    else
```

```
        bit_select <= bit_select + 1'b1;
    end
```

```
always @(*) begin
    case(bit_select)
        3'd0: AN = 8'b1111_1110; // TB0
        3'd1: AN = 8'b1111_1101; // TB1
        3'd2: AN = 8'b1111_1011; // TB2
        3'd3: AN = 8'b1111_0111; // TB3
        3'd4: AN = 8'b1110_1111; // TB4
        3'd5: AN = 8'b1101_1111; // TB5
        3'd6: AN = 8'b1011_1111; // TB6
        3'd7: AN = 8'b0111_1111; // TB7
        default: AN = 8'b1111_1111;
    endcase
end
```

```
// --- 3. 数据选择 ---
```

```
reg [3:0] hex_digit;
always @(*) begin
    case(bit_select)
        3'd0: hex_digit = Data[3:0];
        3'd1: hex_digit = Data[7:4];
        3'd2: hex_digit = Data[11:8];
        3'd3: hex_digit = Data[15:12];
        3'd4: hex_digit = Data[19:16];
        3'd5: hex_digit = Data[23:20];
        3'd6: hex_digit = Data[27:24];
        3'd7: hex_digit = Data[31:28];
        default: hex_digit = 4'h0;
    endcase
end
```

```
// --- 4. 段选解码 (字形显示) ---
```

```
// 依据实验指导书表6.1编码 (共阳极, 0亮1灭)
```

```
// 顺序假定为: A B C D E F G DP (根据 0=00000011 推断 A为最高位)
```

```
always @(*) begin
    case(hex_digit)
        4'h0: SEG = 8'b0000_0011; // 0
        4'h1: SEG = 8'b1001_1111; // 1
        4'h2: SEG = 8'b0010_0101; // 2 (指导书表可能有误, 此处按标准共阳更正: A B D E G亮 ->
00100101)
```

```
// *注意: 如果你需要严格照抄指导书表6.1的编码, 请替换下方: *
```

```
// 实验书表6.1: 2 -> 00100101 (A=0,B=0,C=1,D=0,E=0,F=1,G=0) -> 显示为2
```

```
        4'h3: SEG = 8'b0000_1101; // 3
        4'h4: SEG = 8'b1001_1001; // 4
        4'h5: SEG = 8'b0100_1001; // 5
        4'h6: SEG = 8'b0100_0001; // 6
        4'h7: SEG = 8'b0001_1111; // 7
        4'h8: SEG = 8'b0000_0001; // 8
        4'h9: SEG = 8'b0000_1001; // 9
        4'hA: SEG = 8'b0001_0001; // A
        4'hB: SEG = 8'b1100_0001; // b
```

```

        4'hC: SEG = 8'b0110_0011; // C
        4'hD: SEG = 8'b1000_0101; // d
        4'hE: SEG = 8'b0110_0001; // E
        4'hF: SEG = 8'b0111_0001; // F
        default: SEG = 8'b1111_1111; // 灭
    endcase
end

endmodule

```

(2) 顶层模块 (Seg7_Scan.v)

包含四选一数据选择器逻辑，并实例化 Tube 模块。

```

`timescale 1ns / 1ps

module Seg7_Scan(
    input wire clk_100M,          // 板载时钟
    input wire [0:0] BT,          // 使用 BT[0] 作为复位
    input wire [35:0] SW,         // 36位开关 (31:0为数据, 35:34为选择)
    output wire [7:0] AN,         // 数码管位选
    output wire [7:0] SEG         // 数码管段选
);

wire rst;
assign rst = BT[0]; // 按钮按下拉高复位

// 提取控制信号
wire [1:0] Sel;
assign Sel = SW[35:34]; // 使用最左边的两个开关作为选择位

wire [31:0] Switch_Data;
assign Switch_Data = SW[31:0];

// --- 四选一数据选择器 (Mux) ---
reg [31:0] Data_To_Show;

always @(*) begin
    case(Sel)
        2'b00: Data_To_Show = Switch_Data; // 模式0: 显示开关数据
        2'b01: Data_To_Show = 32'h12345678; // 模式1: 固定常数1
        2'b10: Data_To_Show = 32'h87654321; // 模式2: 固定常数2
        2'b11: Data_To_Show = 32'h19491001; // 模式3: 固定常数3 (国庆)
        default: Data_To_Show = 32'h00000000;
    endcase
end

// --- 实例化数码管显示模块 ---
Tube u_Tube (
    .clk      (clk_100M),
    .rst      (rst),
    .Data     (Data_To_Show),
    .AN       (AN),

```

```
.SEG      (SEG)

);

endmodule
```

(3) 引脚约束文件 (Seg7_Scan.xdc)

直接修改自您提供的 `HCS_A02_ALL.xdc`，我去除了无用的部分并修正了注释，确保能直接跑通。

```
# 时钟信号
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN E3} [get_ports clk_100M]

# 复位信号（映射到 BT[0] 按钮）
set_property IOSTANDARD LVCMOS33 [get_ports {BT[0]}]
set_property PACKAGE_PIN D15 [get_ports {BT[0]}]

# 开关 SW[31:0] 用于数据输入
# 开关 SW[35:34] 用于模式选择 Selection
set_property IOSTANDARD LVCMOS33 [get_ports SW]
set_property PACKAGE_PIN T10 [get_ports {SW[35]}]
set_property PACKAGE_PIN R10 [get_ports {SW[34]}]
# 注意：为了节省篇幅且匹配您的原始文件，此处保留原始管脚分配
# 请确保您的顶层模块端口名 SW 定义为 [35:0] 以匹配此处
set_property PACKAGE_PIN T11 [get_ports {SW[33]}]
set_property PACKAGE_PIN R11 [get_ports {SW[32]}]
set_property PACKAGE_PIN T8 [get_ports {SW[31]}]
set_property PACKAGE_PIN U1 [get_ports {SW[30]}]
set_property PACKAGE_PIN U2 [get_ports {SW[29]}]
set_property PACKAGE_PIN U3 [get_ports {SW[28]}]
set_property PACKAGE_PIN V5 [get_ports {SW[27]}]
set_property PACKAGE_PIN V6 [get_ports {SW[26]}]
set_property PACKAGE_PIN V7 [get_ports {SW[25]}]
set_property PACKAGE_PIN V9 [get_ports {SW[24]}]
set_property PACKAGE_PIN V11 [get_ports {SW[23]}]
set_property PACKAGE_PIN V12 [get_ports {SW[22]}]
set_property PACKAGE_PIN U14 [get_ports {SW[21]}]
set_property PACKAGE_PIN V15 [get_ports {SW[20]}]
set_property PACKAGE_PIN V16 [get_ports {SW[19]}]
set_property PACKAGE_PIN V17 [get_ports {SW[18]}]
set_property PACKAGE_PIN R18 [get_ports {SW[17]}]
set_property PACKAGE_PIN P18 [get_ports {SW[16]}]
set_property PACKAGE_PIN R8 [get_ports {SW[15]}]
set_property PACKAGE_PIN T9 [get_ports {SW[14]}]
set_property PACKAGE_PIN V1 [get_ports {SW[13]}]
set_property PACKAGE_PIN V2 [get_ports {SW[12]}]
set_property PACKAGE_PIN V4 [get_ports {SW[11]}]
set_property PACKAGE_PIN U6 [get_ports {SW[10]}]
set_property PACKAGE_PIN U7 [get_ports {SW[9]}]
set_property PACKAGE_PIN U9 [get_ports {SW[8]}]
set_property PACKAGE_PIN U11 [get_ports {SW[7]}]
set_property PACKAGE_PIN U12 [get_ports {SW[6]}]
set_property PACKAGE_PIN U13 [get_ports {SW[5]}]
set_property PACKAGE_PIN V14 [get_ports {SW[4]}]
```

```

set_property PACKAGE_PIN U16 [get_ports {SW[3]}]
set_property PACKAGE_PIN U17 [get_ports {SW[2]}]
set_property PACKAGE_PIN T18 [get_ports {SW[1]}]
set_property PACKAGE_PIN P17 [get_ports {SW[0]}]

```

数码管段选 SEG[7:0]

```

set_property IOSTANDARD LVCMOS33 [get_ports SEG]
set_property PACKAGE_PIN E2 [get_ports {SEG[7]}]
set_property PACKAGE_PIN A3 [get_ports {SEG[6]}]
set_property PACKAGE_PIN B1 [get_ports {SEG[5]}]
set_property PACKAGE_PIN E1 [get_ports {SEG[4]}]
set_property PACKAGE_PIN F1 [get_ports {SEG[3]}]
set_property PACKAGE_PIN D2 [get_ports {SEG[2]}]
set_property PACKAGE_PIN B3 [get_ports {SEG[1]}]
set_property PACKAGE_PIN C1 [get_ports {SEG[0]}]

```

数码管位选 AN[7:0]

```

set_property IOSTANDARD LVCMOS33 [get_ports AN]
set_property PACKAGE_PIN G1 [get_ports {AN[7]}]
set_property PACKAGE_PIN B2 [get_ports {AN[6]}]
set_property PACKAGE_PIN A1 [get_ports {AN[5]}]
set_property PACKAGE_PIN B4 [get_ports {AN[4]}]
set_property PACKAGE_PIN A4 [get_ports {AN[3]}]
set_property PACKAGE_PIN A5 [get_ports {AN[2]}]
set_property PACKAGE_PIN B6 [get_ports {AN[1]}]
set_property PACKAGE_PIN A6 [get_ports {AN[0]}]

```

(4) 仿真激励代码 (tb_Seg7_Scan.v)

用于验证逻辑是否正确。

```

`timescale 1ns / 1ps

module tb_Seg7_Scan;

    // Inputs
    reg clk_100M;
    reg [0:0] BT;
    reg [35:0] SW;

    // Outputs
    wire [7:0] AN;
    wire [7:0] SEG;

    // Instantiate the Unit Under Test (UUT)
    Seg7_Scan uut (
        .clk_100M(clk_100M),
        .BT(BT),
        .SW(SW),
        .AN(AN),
        .SEG(SEG)
    );

```

```
initial begin
    // Initialize Inputs
    clk_100M = 0;
    BT = 0;      // 复位，假设按下为1
    SW = 0;

    // 1. 复位测试
    #100;
    BT[0] = 1;   // 按下复位
    #100;
    BT[0] = 0;   // 释放复位

    // 2. 测试模式 1: 显示常数 12345678
    SW[35:34] = 2'b01;
    #5000000; // 仿真运行足够长的时间看 AN 切换 (5ms)

    // 3. 测试模式 0: 显示开关数据 AAAAAAAAA
    SW[35:34] = 2'b00;
    SW[31:0] = 32'hAAAAAAAA;
    #5000000;

    $stop;
end

// 产生 100MHz 时钟 (周期 10ns)
always #5 clk_100M = ~clk_100M;

endmodule
```

2. 严谨解析 (Technical Explanation)

1. 动态扫描原理 (Dynamic Scanning):

- 由于硬件只有一组段选信号 (SEG)，8 个数码管共用这组信号线。为了显示不同的数字，必须利用人眼的视觉暂留 (Persistence of Vision) 效应。
- 代码中利用 `clk_fresh` (约381Hz) 作为刷新时钟，每隔约2.6毫秒 (ms) 切换一次位选 `AN`。在这个极短的时间窗口内，只选通一个数码管 (AN置低)，并输出该数位对应的数据 (SEG)。
- 因为刷新频率 (>50Hz) 远快于人眼反应速度，看起来8个数码管是同时常亮的。

2. 共阳极与逻辑电平 (Common Anode Logic):

- HCS-A02 板卡的数码管驱动经过了三极管反相。FPGA 输出低电平 (Logic 0) 到 AN 引脚，经过板上电路反相驱动，使数码管共阳极获高电平，从而选中数码管。
- 段选 SEG 同样是低电平有效 (Active Low)，当 FPGA 输出 0 时，LED 内部二极管导通点亮。
- 代码中的编码如 `8'b0000_0011` 对应数字0，意味着 A~F 段为0 (亮)，G和DP段为1 (灭)。

3. 资源映射与复用 (Constraint Mapping):

- 数据输入:** 实验利用了 `SW[31:0]` 这个宽位宽直接映射32位数据，满足实验中“在板子上跑通”且能显示任意数据的需求。
- 控制位复用:** 因为数据占用了32位，选择信号 `sel` 复用了高位的 `SW[35:34]`，这是一种常见的 FPGA

实验技巧，避免了按键抖动处理的麻烦，且节省了 IO 资源。

3. 精准概括 (Summary)

本实验通过 **分时复用 (Time Division Multiplexing)** 技术，利用计数器产生 kHz 级扫描时钟，快速轮流选通8个数码管并送入对应的段码，从而在有限的 IO 引脚上实现了32位数据的**动态显示**。

User

ok, 请教我如何在板子上进行实验演示

Assistant

技术回答：如何在板子上进行实验演示

在 Vivado 中生成比特流 (Generate Bitstream) 并下载 (Program Device) 均显示成功 (Complete) 后，请按照以下步骤操作 HCS-A02 实验板进行演示。

1. 实例先行 (Demo Steps)

请严格按照以下顺序操作开关和按键，验证实验现象：

第一步：系统复位

- 操作：**按下板子上的 **BT0** 按钮（对应代码中 `BT[0]`），然后松开。
- 现象：**系统复位，扫描计数清零（虽然肉眼看不出明显变化，但这是标准操作）。

第二步：验证固定常数显示 (Mode 1, 2, 3)

这一步用来验证数码管扫描逻辑是否正常工作。请操作 **SW35** 和 **SW34** 号开关（位于最左侧）。

- **模式 1：顺序序列**
 - **操作：**将 **SW35** 拨到 **下 (0)**，**SW34** 拨到 **上 (1)**。（即 `Se1 = 01`）
 - **现象：**8个数码管稳定显示：`12345678`。
- **模式 2：逆序序列**
 - **操作：**将 **SW35** 拨到 **上 (1)**，**SW34** 拨到 **下 (0)**。（即 `Se1 = 10`）
 - **现象：**8个数码管稳定显示：`87654321`。
- **模式 3：国庆/生日数字**
 - **操作：**将 **SW35** 拨到 **上 (1)**，**SW34** 拨到 **上 (1)**。（即 `Se1 = 11`）
 - **现象：**8个数码管稳定显示：`19491001`。

第三步：验证手动开关输入 (Mode 0)

这一步验证你是否能通过32个开关控制32位数据。

1. 进入模式 0：

- 操作：将 SW35 和 SW34 都拨到 下 (0)。 (即 Sel = 00)
- 现象：此时数码管显示的数值由右边的开关 SW[31:0] 决定。

2. 控制最低位数码管 (最右边)：

- 原理：数码管显示的是16进制。每4个开关控制1个数码管 (8421码)。
- 操作：拨动 SW0, SW1, SW2, SW3。
- 现象：
 - 仅拨上 SW0 -> 最右数码管显示 1。
 - 仅拨上 SW1 -> 最右数码管显示 2。
 - SW0 和 SW3 同时拨上 (二进制 1001) -> 最右数码管显示 9。
 - SW0 ~ SW3 全部拨上 (二进制 1111) -> 最右数码管显示 F。

3. 控制最高位数码管 (最左边)：

- 操作：操作 SW28 至 SW31。
- 现象：最左侧的数码管随之改变。

2. 严谨解析 (Technical Explanation)

1. 比特流下载 (Bitstream Configuration)：

- 你生成的 .bit 文件是 FPGA 的配置数据。当你点击 "Program Device" 时，PC 通过 JTAG 接口将这些二进制流烧录到 FPGA 的 SRAM 层中。因为 SRAM 掉电易失，所以如果你切断电源，再次上电需要重新下载。

2. I/O 映射验证 (I/O Mapping Verification)：

- 实验中 SW[35:34] 作为多路复用器 (Mux) 的选择端 (Select Line)。FPGA 内部逻辑根据这两个引脚的电平高低 (3.3V 为逻辑1, 0V 为逻辑0)，实时切换输入的数据源 Data_To_Show。
- 由于 Verilog 中使用的是组合逻辑 (always @(*))，只要你拨动开关，数据路径会立发生改变并在下一个扫描周期 (约2ms内) 立刻反映在数码管上，人眼感觉是“实时”的。

3. 16进制与二进制对应关系 (Hex to Binary)：

- 在 Mode 0 演示中，你需要理解 4位二进制 = 1位十六进制 的关系。
 - 例如：若要让某一位显示字母 A (十六进制的10)，你需要将对应的4个开关拨成 1010 (即：上、下、上、下)。
-

3. 精准概括 (Summary)

操作演示的核心在于利用 **SW35/34** 切换四种显示模式，确认固定数字无误后，切换至模式0，通过拨动 **每4个一组的开关** 验证其对数码管 **0-F 字形** 的实时控制。